楽天ショッピングツール — 子役カウンターカチカチくん監視システム

作成日: 2025-07-26 (JST)\ 作成者: ChatGPT (o3) / Nana Nagai プロジェクトチーム

---

1. 目的と概要

このプロジェクトは、楽天市場ショップ **「子役カウンターカチカチくん」** の新商品および再販商品を自動検知し、即時に Discord へ通知する監視システムを構築・運用することを目的としています。サーバーは専用の Ubuntu Server 24.04 LTS マシン上で常時稼働し、監視ジョブは `cron` により定期実行されます。

2. フェーズ進捗サマリー

フェーズ	目標	達成状況
Phase 0	要件定義・技術選定	✅完了
Phase 1	HTML/API 構造調査・PoC 実装・DB 設計	✅完了
Phase 2	本実装・Git / CI 基盤構築	✅完了 (2025-07-25)
Phase 3	運用自動化／監視強化	✅コア機能完了 (2025-07-26)

3. 主な達成事項

1. **サーバー環境構築**
2. Ubuntu Server 24.04 LTS をクリーンインストール
3. SSH 公開鍵認証設定 (Windows 11 Pro 端末から `OpenSSH` クライアントで接続)
4. Python 3.12, `pipx`, `venv`, `poetry` を導入し依存管理を統一
5. 必要パッケージ: `requests`, `beautifulsoup4`, `python-dotenv`, `discord-webhook`, `schedule` ほか
6. **Git リポジトリ整備**
7. リモート: `ssh://myserver/~/rakutenApp.git`
8. ブランチ戦略: `main` (保守) ／ `dev` (開発)
9. `pre-commit` hook による `ruff` ・ `black` フォーマッタ自動実行
10. Windows ↔ Server 間で `git push/pull` が確認済み
11. **監視スクリプト実装 (``)**
12. **商品一覧ページ** (`https://item.rakuten.co.jp/auc-p-entamestore/c/0000000174/?s=4#risFil`) を HTML 取得
13. 各商品の `itemCode` とタイトルを抽出
14. **新規**: `itemCode` が過去 DB に存在しない場合
15. **再販**: `itemCode` は同一だがタイトル末尾の日付や発送月が更新された場合
16. SQLite DB で前回状態を保持し差分を検出
17. **Discord 通知連携**
18. Webhook URL を `.env` で管理 (`DISCORD_WEBHOOK_URL`)
19. 新検出時に埋め込みメッセージで即時送信 (商品名・URL・判定区分・検出時刻)
20. **ジョブスケジューリング**

21. `cron` → `*/5 * * * * /usr/bin/python3 /opt/rakutenApp/rakuten_watcher.py >> /var/log/rakuten_watcher.log 2>&1`
22. ローテーション: `logrotate.d/rakuten_watcher` を設定
23. **ロギング & エラーハンドリング**
24. `logging` モジュールで INFO/ERROR を出力
25. ネットワーク例外時に指数バックオフでリトライ

4. フォルダ / ファイル構成 (抜粋)

```
/opt/rakutenApp/
├── .env                  # 機密設定 (Webhook 等)
├── rakuten_watcher.py    # 監視メインスクリプト
├── db/
│   └── products.sqlite   # 差分判定用 DB
├── utils/
│   ├── parser.py         # HTML 解析ロジック
│   └── notifier.py       # Discord 送信モジュール
├── tests/
│   └── test_parser.py    # 単体テスト (pytest)
└── README.md             # セットアップ & 運用手順
```

5. 環境変数 & シークレット

変数	用途	設定例
`DISCORD_WEBHOOK_URL`	通知先 Webhook	https://discord.com/api/webhooks/…
`USER_AGENT`	スクレイピング時の UA	Mozilla/5.0 (compatible; RakutenWatcher/1.0)

注: `.env` は `.gitignore` 済み。サーバー側で `systemd` の `EnvironmentFile` に指定すると安全。

6. 運用 / デプロイ手順

1. **初期セットアップ (新サーバー)**

```
sudo apt update && sudo apt install -y git python3-pip logrotate
git clone ssh://myserver/~/rakutenApp.git /opt/rakutenApp
cd /opt/rakutenApp && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # 値を編集
```

2. **手動テスト**

```
python rakuten_watcher.py --dry-run
```

3. **cron 登録**\ `crontab -e` で前述のジョブを追加

7. 残タスク (Phase 3 仕上げ)

#	内容	優先度	メモ
1	ruff 警告完全解消 (残 5 件)	★★★	`ruff --fix` + 手動対応
2	`requests_mock` テスト設定の調整	★★☆	CI で安定実行・外部通信排除
3	README へ Grafana / Cloudflare 運用ガイド追記	★★☆	ダッシュボード JSON, FAQ 反映
4	Playwright E2E テスト拡充	★☆☆	レイアウト変更リグレッション検知

8. 既知の課題 / リスク 既知の課題 / リスク

- **HTML レイアウト変更** に弱い → セレクタ変更時は `parser.py` を修正。
- **API が公式に存在する場合** → 後日逆解析予定 (優先度低)。
- サーバー障害時の **自動復旧** (systemd restart) は設定済みだが、ネットワーク断続時は要観察。

9. 今後 2 週間のアクション (TL;DR)

1. Cloudflare 対策調査 & 実装
2. Prometheus Node Exporter 導入、Grafana ダッシュボード作成
3. Discord ボットコマンド `!interval 10` 実装 (監視分解能変更)
4. README を英語化し OSS 化の是非を検討

10. 引き継ぎポイント

- **コード品質:** `ruff + black` で統一。コミット前に `pre-commit run --all-files` を徹底。
- **機密情報:** `.env` は共有しない。必要なら 1Password/Bitwarden 経由で渡す。
- **連絡先:** Nana Nagai (Discord: Nana#1234) / Git サーバー管理者: yang_server\@LAN

本書は自由に複製・改変可。後続チームは必要に応じて加筆してください。